CO1

AT2

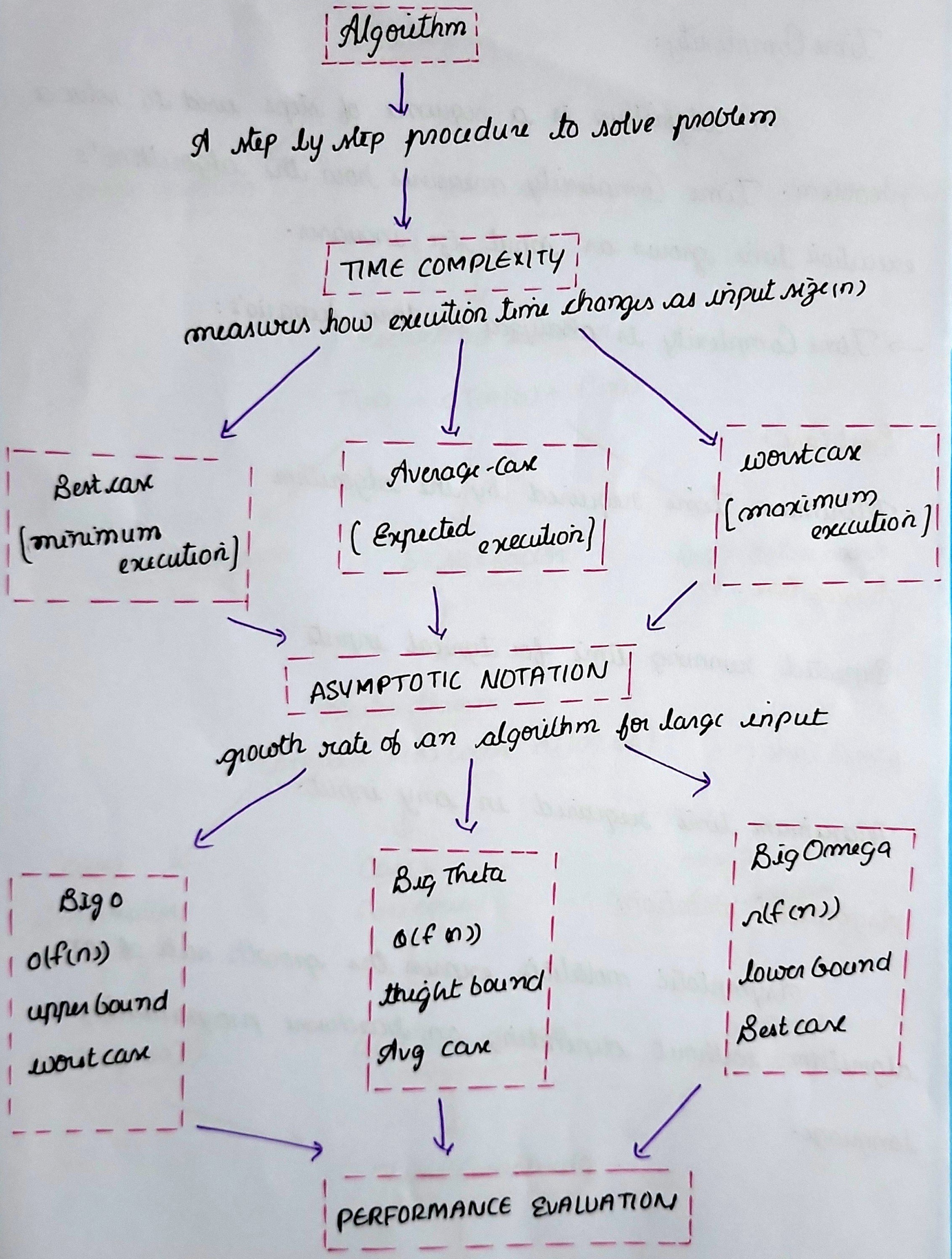
Name : K.NAVYA

Reg no : 192371022

Course : CSA0624

Data Analysis and Algorithms

1)


Algorithm
A step by step procedure to solve problem
TIME COMPLEXITY
measures how execution time changes as input size (n)
Best case
(minimum execution)
Average-case
(Expected execution)
Worst case
(maximum execution)
ASYMPTOTIC NOTATION
growth rate of an algorithm for large input
Big O
O(f(n))
upper bound
worst case
Big Theta
Θ(f(n))
thight bound
Avg case
Big Omega
Ω(f(n))
lower bound
Best case
PERFORMANCE EVALUATION

Time Complexity:

An algorithm is a sequence of steps used to solve a problem. Time Complexity measures how the algorithm's execution time grows as input size increases.

→ Time Complexity is analyzed in three scenario's:

Best Case:

Minimum time required by the algorithm

Average Case:

Expected running time for typical inputs

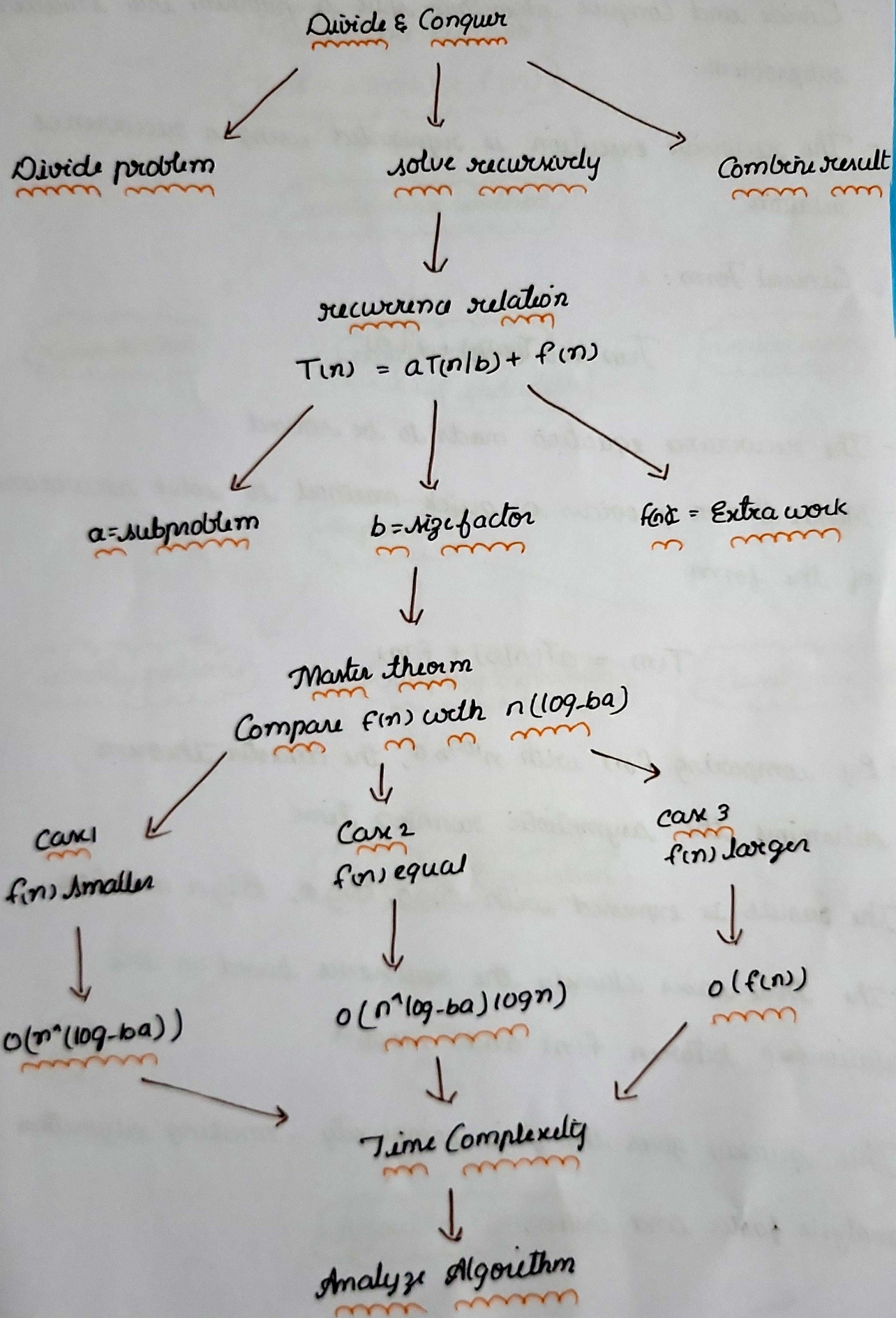
Worst Case:

Maximum time required in any input.

Asymptotic notation:

Asymptotic notation expresses the growth rate of an algorithm without depending on hardware programming language.

2)


Divide & Conquer
Divide problem
solve recursively
Combine result
recurrence relation
T(n) = aT(n/b) + f(n)
a = subproblem
b = size factor
f(n) = Extra work
Master theorm
Compare f(n) with n(log-ba)
Case 1
f(n) smaller
Case 2
f(n) equal
Case 3
f(n) larger
O(n^(log-ba))
O(n^log-ba log n)
O(f(n))
Time Complexity
Analyze Algorithm

- Divide and Conquer algorithm split a problem into smaller subproblem.
- The recursive execution is represented using a recurrence relation

General Form:

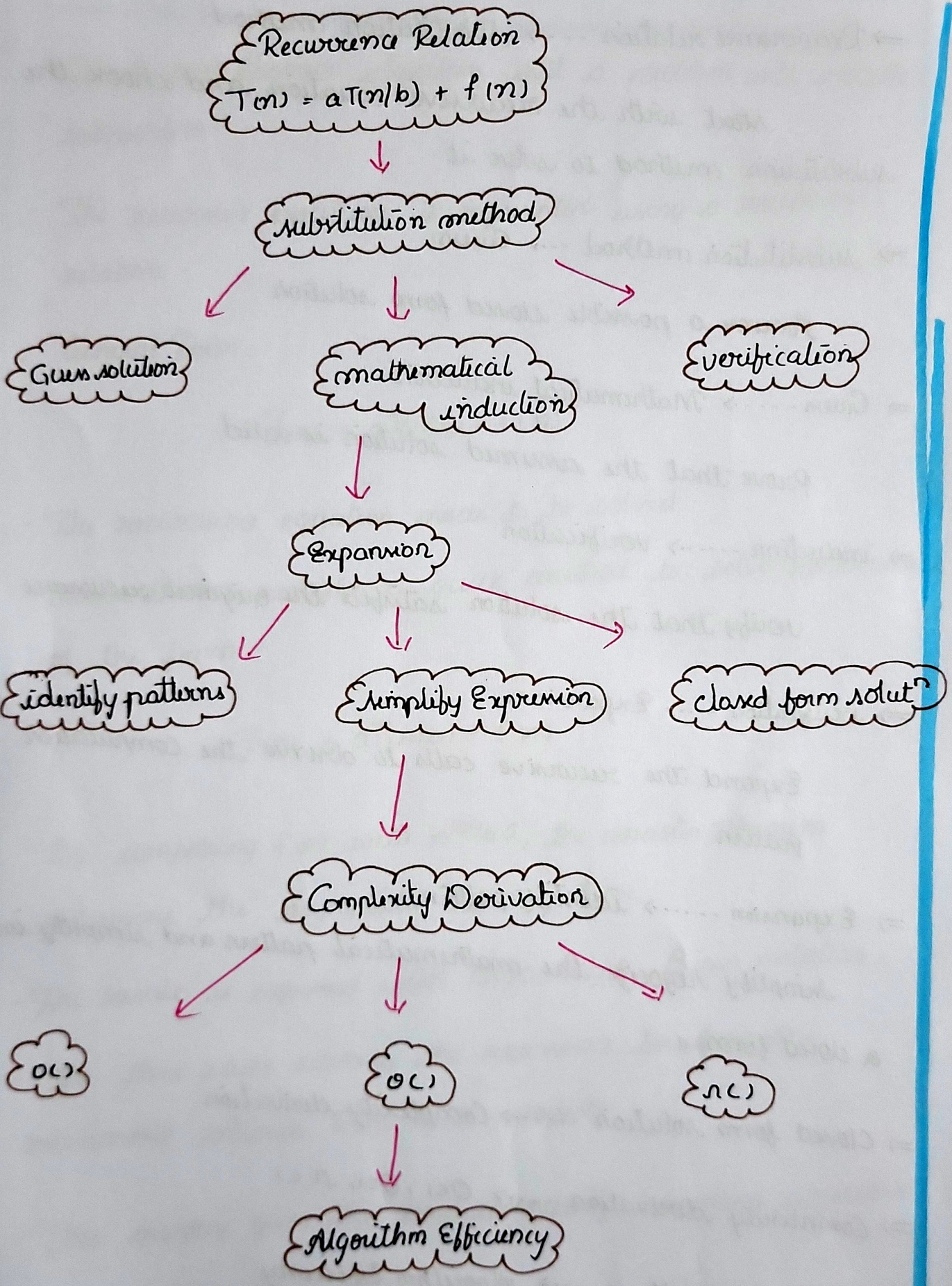
$$T(n) = aT(n/b) + f(n)$$

- The recurrance equation needs to be solved
- Master theorm provides a quick method to solve recurrance of the form

$$T(n) = aT(n/b) + f(n)$$

- By comparing $f(n)$ with $n^{\log_b a}$, the master theorem determines the asymptiotic running time
- The result is expressed with Big O, Big Θ, Big Ω notation.
- The three cases classify the recurrence based on the relationship between $f(n)$ and $n^{\log_b a}$
- This quickly gives the time complexity, making algorithm analysis faster and easier.

3)


Recurrence Relation
T(n) = aT(n/b) + f(n)
substitution method
Guess solution
mathematical induction
verification
Expansion
identify patterns
simplify Expression
closed form solut?
Complexity Derivation
O( )
O( )
n( )
Algorithm Efficiency

⇒ Recurrance relation ---> Substitution method

Start with the recursive equation and choose the substitution method to solve it

⇒ Substitution method ---> Guess

Assume a possible closed form solution

⇒ Guess ----> Mathematical induction

Prove that the assumed solution is valid

⇒ induction -----> verification

Verify that the solution satisfies the original recurrence

⇒ verification ----> Expansion

Expand the recursive calls to observe the computation pattern

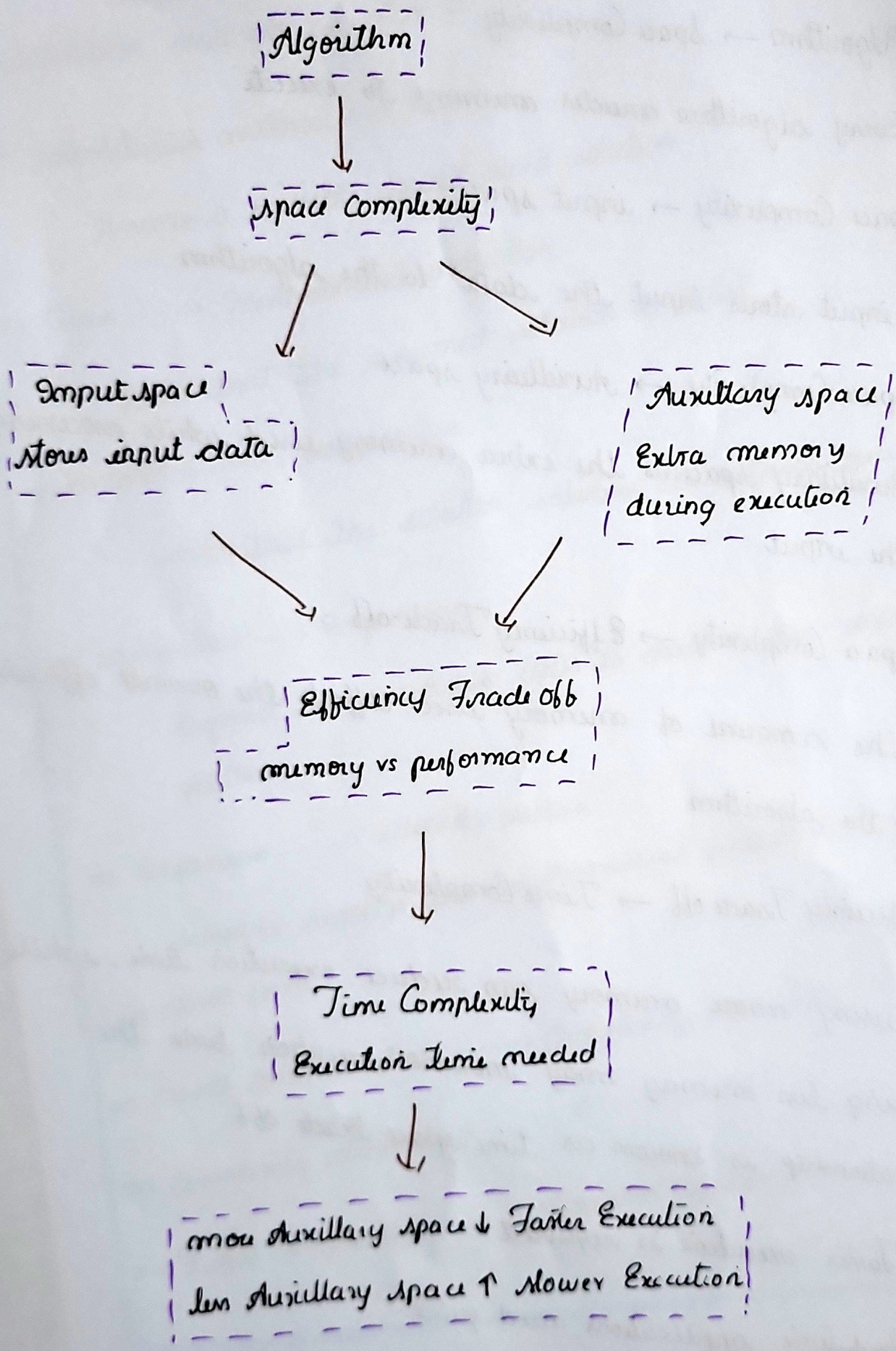
⇒ Expansion -----> identify pattern

Simplify regocize the mathematical pattern and simplify int a closed form

⇒ Closed form solution -------> Complexity derivation

⇒ Complexity derivation ----> O(1), O(n), Ω(1)

⇒ Final Complexity -------> Algorithm Efficiency


Algorithm
Space Complexity
Input space
Stores input data
Auxiliary space
Extra memory
during execution
Efficiency Trade off
memory vs performance
Time Complexity
Execution time needed
more Auxillary space ↓ Faster Execution
less Auxillary space ↑ Slower Execution

Algorithm → Space Complexity

⇒ Every algorithm needs memory to execute

Space Complexity → input space

⇒ input stores input the data to the algorithm

Space Complexity → Auxilliary space

⇒ Auxilliary space is the extra memory used while processing the input

Space Complexity → Efficiency Trade-off

⇒ The amount of memory used affects the overall efficiency of the algorithm

Efficiency Trade off → Time Complexity

⇒ using more memory can reduce execution time, while using less memay may increase execution time the relationship is known as time-space trade off

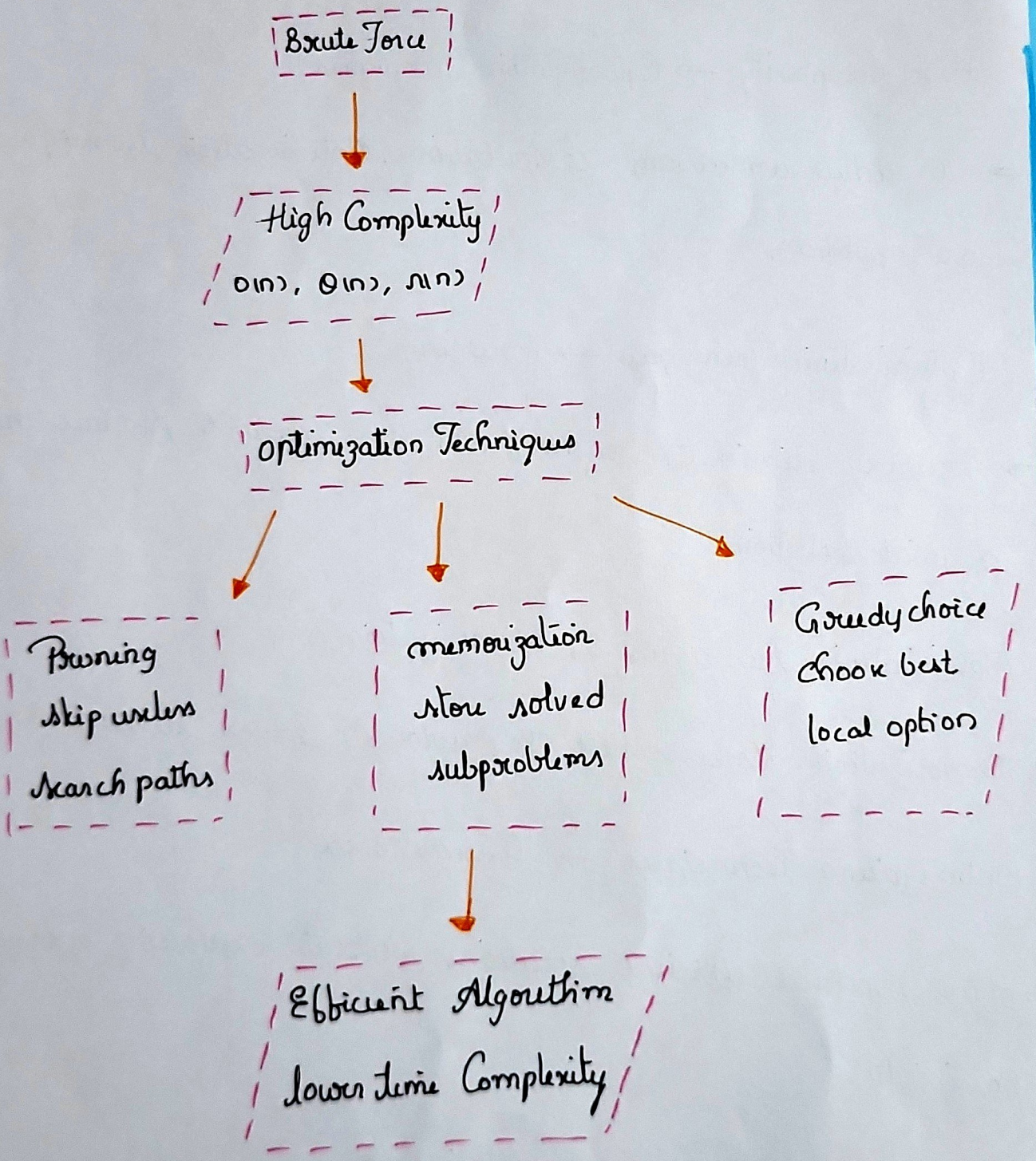
⇒ Faster execution is required

⇒ Real time applications need quick.

6) Brute Force → High Complexity → Optimization Techniques

↓

Efficient Algorithm



Brute Force ⟶ High Complexity

⇒ This leads to high time Complexity, making them inefficient for larger input

High Complexity ⟶ Optimization Techniques

⇒ To reduce unnecessary computations, Optimization Techniques are applied.

Optimization Techniques ⟶ Pruning

⇒ Pruning eliminates search paths that cannot produce the optimal solution

Optimization Techniques ⟶ Memorization

⇒ Memorization stores result of previously solved subproblem.

Optimization Techniques ⟶ Greedy choice

⇒ often produces efficient solutions without exploring every possibility

⇒ Optimization ⟶ Efficient Algorithm